

FLUID Engineering Upgrade Plan

Purpose

This document defines how FLUID should evolve from a Chrome side-panel browser assistant into a practical engineering assistant that can investigate, fix, and verify real WordPress/support issues with evidence.

The target is not to copy hidden reasoning. The target is to copy the engineering workflow:

1. Understand the user goal.
2. Collect real evidence from browser, HTTP, files, logs, and tests.
3. Decide the smallest safe fix.
4. Apply changes only through the correct surface.
5. Verify the result with repeatable checks.
6. Save the lesson so the same failure does not repeat.

Current State

FLUID currently has these working parts:

- Chrome sidebar extension in 'extensions/chrome-sidebar'.
- Local Node bridge in 'tools/sidebar-bridge-node'.
- Task checkpoint storage in 'runs/bridge-tasks'.
- Attachments storage in 'runs/bridge-attachments'.
- Provider contract that can return 'reply', 'reasoning_summary', 'questions', 'actions', 'done', and 'needs_approval'.
- Local planner in 'tools/sidebar-bridge-node/src/local-planner.mjs'.
- Provider router in 'tools/sidebar-bridge-node/src/reasoner.mjs'.
- Codex provider in 'tools/sidebar-bridge-node/src/codex-provider.mjs'.
- Unit tests in 'tools/sidebar-bridge-node/test'.

Current limitation:

- The Chrome extension is good at page observation and browser actions.
- The local planner is too keyword-driven for complex support work.
- The bridge does not yet own full engineering tools such as filesystem inspection, shell commands, lint/test runs, Git checks, WordPress HTTP sessions, or structured verification.
- It can act in the browser, but it cannot yet reliably do full root-cause investigation without help.

Extension Vs Desktop Decision

Do not move everything into the Chrome extension.

Use a hybrid architecture:

“text

Chrome Sidebar

- user interface
- current tab snapshot
- screenshots

- safe browser actions

Local Desktop Bridge

- planning and task state
- shell/file/git tools
- WordPress HTTP session tools
- code patch and backup workflow
- lint/test/verification
- memory and evals

Future Python Core

- agent orchestration
- memory retrieval
- eval scoring
- PR-gated self improvement

““

Reason:

Chrome extensions are sandboxed. They should not directly edit local files, run shell commands, lint PHP, inspect Git history, or deploy changes. The bridge can do those things with explicit safety gates.

Main Design Rule

FLUID must never jump from user text directly to a fix.

Every non-trivial task must follow this loop:

““text

Goal -> Evidence -> Hypothesis -> Fix Plan -> Approval Gate -> Patch -> Verification -> Lesson

““

If evidence is missing, FLUID must ask a short question or run a safe observation step.

Required Work Modes

1. Browser Mode

Use when the task is only browser navigation or visible page checking.

Allowed actions:

- navigate
- click
- type
- press
- wait
- observe
- screenshot

Browser Mode must stop before:

- publishing
- deleting
- resetting
- payment
- changing settings
- submitting real data
- editing live code

2. Investigation Mode

Use for support issues like:

- "search returns no results"
- "button hidden"
- "field missing"
- "dropdown not working"
- "responsive issue"
- "CSV export missing field"
- "booking does not show"

Investigation Mode collects:

- current URL and page snapshot
- request payload or query string
- console/network clues when available
- active theme/plugin surface
- WordPress REST/API evidence when available
- relevant source code path or theme file
- exact reproducible condition

No code change is allowed until the root cause is stated in a short evidence summary.

3. Engineering Fix Mode

Use after root cause is known.

Required steps:

1. Identify target surface:
 - child theme
 - mu-plugin
 - Code Snippets
 - plugin repo
 - local package
2. Back up existing file or record Git status.
3. Apply smallest patch.
4. Run syntax/test check.
5. Verify in browser or HTTP.
6. Save final explanation.

4. Review-Only Mode

Use when the user asks for:

- "list first"
- "audit"
- "explain"
- "logic only"
- "do not implement yet"

In this mode FLUID must create notes and avoid code changes.

5. Learning/Eval Mode

Use after a failure or after a successful fix.

The system saves:

- task type
- root cause
- evidence used
- wrong path avoided
- correct fix surface
- verification commands
- future regression test idea

Anti-Hallucination Rules

FLUID must not say a fix is done unless it has one of these:

- live page verification
- HTTP response verification
- screenshot verification
- test output
- lint output
- Git diff review
- WordPress admin/file editor confirmation

FLUID must not guess:

- active theme
- plugin availability
- file path
- hook name
- shortcode behavior
- database setting
- REST route behavior
- submitted form fields

When unsure, FLUID should say:

“text

I do not have evidence yet. I will inspect X first.

““

Real Failure Lessons To Encode

Case 1: Safety Wording Misrouted To WP Reset

Observed failure:

- User said "do not publish/delete/reset".
- Local planner saw 'reset' and routed to WP Reset.

Fix already started:

- 'isResetOnlySafetyBoundary()' added in 'tools/sidebar-bridge-node/src/local-planner.mjs'.
- Unit test added in 'tools/sidebar-bridge-node/test/local-planner.test.mjs'.

Permanent rule:

- Negative safety phrases must not become action intent.
- "do not reset" means block/reset avoidance, not reset execution.

Acceptance test:

““text

Goal: "Investigate this WordPress issue. Do not publish/delete/reset anything."

Expected: no WP Reset route.

““

Case 2: WordPress Hidden Login Route

Observed issue:

- Direct '/wp-login.php' failed on one site.
- Valid login started at '/dashboard', which redirected to a special login URL.

Permanent rule:

- If the user provides an admin/dashboard URL, prefer that route over guessing '/wp-login.php'.
- If direct login fails, inspect redirects before declaring login impossible.

Acceptance test:

““text

Input URL: https://example.com/dashboard

Expected: preserve user-provided login route and follow redirects.

““

Case 3: Directorist Geo Search No Results

Observed issue:

- Geolocation filled the visible address.

- Page had multiple Directorist search forms with duplicate 'cityLat/cityLng'.
- Only the first coordinate pair was updated.
- Submitted form could send address with blank coordinates.

Correct workflow:

1. Inspect live form fields.
2. Inspect Directorist query condition.
3. Verify published listings with known coordinates.
4. Patch child theme, not core plugin.
5. Remove old orphan geolocation code.
6. Verify logged-out page and search results.

Permanent rule:

- For search bugs, inspect actual request fields before touching code.
- For repeated forms, check duplicate IDs and submitted form scope.

Acceptance test:

““text

Given multiple search forms

When one form has address text and another has known cityLat/cityLng

Then the submitted form gets cityLat/cityLng before submit.

““

Case 4: Draft Test Listing Misleading Search Result

Observed issue:

- User supplied a preview URL for listing '3862'.
- The listing was draft.
- Public search should not return draft content.

Permanent rule:

- Before using a test listing for public search verification, check status.
- If it is draft/private/pending, verify against a published listing too.

Case 5: Booking Dropdown Regression

Observed issue family:

- A previous fix was made for one broken booking condition.
- A later change risked disrupting original booking dropdown behavior.

Permanent rule:

- Before editing booking code, inspect the old fix, plugin state, and a comparable working local flow.
- Preserve original plugin behavior unless the defect is proven.

Implementation Phases

Phase 0: Freeze And Document Current Behavior

Goal:

Create a stable baseline before adding power tools.

Tasks:

- Run current test suite.
- Save current planner behavior examples.
- Document the existing bridge API.
- Record current Chrome sidebar action loop.
- Commit the already-made WP Reset safety patch after review.

Files:

- 'tools/sidebar-bridge-node/src/local-planner.mjs'
- 'tools/sidebar-bridge-node/test/local-planner.test.mjs'
- 'docs/CHROME_SIDEBAR_BRIDGE.md'

Acceptance:

- 'npm test --prefix tools/sidebar-bridge-node' passes.
- Known WP Reset false-positive test passes.
- No behavior change outside planner safety.

Phase 1: Task Classifier

Goal:

Replace fragile keyword-only routing with an intent classifier.

Add a new module:

```
““text
tools/sidebar-bridge-node/src/task-classifier.mjs
““
```

Classifier output:

```
““json
{
  "task_type": "browser_only | investigation | engineering_fix | review_only | risky_action |
unknown",
  "domain": "wordpress | directorist | generic_web | github | local_repo | unknown",
  "risk_level": "safe | needs_confirmation | blocked",
  "evidence_needed": ["page_snapshot", "http_request", "file_read"],
  "reason": "short visible reason"
}
““
```

Rules:

- Negative words like 'do not', 'don't', 'never', 'without asking' must reduce action intent.
- User-provided URLs must be preserved.
- Browser-only tasks should remain fast.
- Complex bug reports should route to Investigation Mode.

Tests:

- "open google" -> browser_only.
- "check why search no result" -> investigation.
- "fix in child theme" -> engineering_fix but only after evidence.
- "do not reset" -> not WP Reset.
- "reset this local site" -> risky_action, needs confirmation.

Phase 2: Evidence Store

Goal:

Make every task traceable.

Add:

```
““text
tools/sidebar-bridge-node/src/evidence-store.mjs
““
```

Store under:

```
““text
runs/bridge-tasks/{task_id}/evidence/
““
```

Evidence types:

- page snapshot
- screenshot path
- URL
- HTTP response summary
- request payload/query string
- file path and hash
- command output summary
- patch path
- verification result

Task checkpoint should include:

```
““json
{
  "evidence": [
    {
```



```

    "type": "http",
    "title": "Search endpoint with Shrewsbury coordinates",
    "summary": "Returned OATH COFFEE, Kal, Peaberry",
    "path": "runs/bridge-tasks/.../evidence/search-result.json",
    "created_at": "..."
  }
]
}
""

```

Acceptance:

- Every investigation task shows an evidence list in the sidebar trace.
- Final answer can cite the evidence labels.

Phase 3: Desktop Tool Adapter

Goal:

Give the bridge safe local engineering hands.

Add a tool registry:

```

""text
tools/sidebar-bridge-node/src/tool-registry.mjs
""

```

Current safe evidence tools:

- 'http_get'
- 'wp_active_theme'
- 'wp_plugins'
- 'wp_overview'
- 'file_read'
- 'file_search'
- 'git_status'
- 'git_diff_summary'
- 'php_lint'
- 'node_test'
- 'chrome_cdp_check'

Current approval-gated file-change workflow:

- 'file_change_prepare'
- 'file_change_check'
- 'file_change_apply'

Safety:

- File writes require target path under allowed workspace or explicit user approval.
- Generic shell/deploy/database/payment/write-like tool names are blocked before execution.
- Local tools run without a shell, with bounded output and timeouts.

- Local file paths outside the configured workspace are rejected.
- Patch workflow prepare creates a backup and staged patched copy first.
- Patch workflow apply requires 'approved: true' and refuses stale applies if the target changed after backup.

Acceptance:

- Tool calls are logged into task evidence.
- Risky tool requests ask the user before execution.

Phase 4: WordPress Support Toolkit

Goal:

Make Directorist/WordPress debugging repeatable.

Add:

```
""text
tools/sidebar-bridge-node/src/wordpress-tools.mjs
""
```

Capabilities:

- login with cookies
- follow redirects
- read admin pages
- read active theme from Themes page
- read theme editor file
- submit theme editor file
- check Code Snippets availability
- query WordPress REST
- detect Directorist directories
- detect listing status/meta through REST/admin HTML
- run logged-out page checks

Prebuilt investigations:

- Directorist search/radius bug
- Directorist add-listing validation bug
- Directorist booking display bug
- Directorist CSV/export field bug
- responsive child-theme CSS bug

Acceptance:

- Given WP credentials and a page URL, FLUID can produce:
 - active theme
 - relevant plugin/script list
 - request payload
 - likely fix surface
 - verification plan

Phase 5: Patch Workflow

Goal:

Make live changes reviewable and rollback-safe.

Workflow:

1. Read current file.
2. Save local backup.
3. Create patched local copy.
4. Run syntax check.
5. Submit through correct channel.
6. Re-read live file.
7. Verify public page.

Required metadata:

```
""json
{
  "target": "yourspace.global dRestaurant Child functions.php",
  "backup": "C:/Users/Arafat/Documents/support/...",
  "patched_copy": "C:/Users/Arafat/Documents/support/...",
  "syntax_check": "php -l passed",
  "live_verify": "public page contains new script"
}
""
```

Acceptance:

- No live file edit without backup.
- No PHP 'functions.php' edit without 'php -l'.
- No final "fixed" without verification evidence.

Phase 6: Browser DevTools Verification

Goal:

Verify behavior beyond static HTML.

Add a Chrome DevTools Protocol helper:

```
""text
tools/sidebar-bridge-node/src/chrome-cdp.mjs
""
```

Use cases:

- emulate geolocation
- inspect DOM values after click
- check button visibility

- run mobile viewport check
- capture screenshot
- inspect console errors
- inspect network requests

Acceptance:

- The tool can open a URL, set viewport, set geolocation, run a JS assertion, and save screenshot/evidence.

Phase 7: Memory And Evals

Goal:

Turn every failure into a regression test.

Add:

```

"text
src/arafatai/evals/support_cases/
tools/sidebar-bridge-node/test/support-cases.test.mjs
"

```

Case format:

```

"json
{
  "id": "directorist-geo-duplicate-fields",
  "goal": "Geo locating search returns no results",
  "given": {
    "forms": "multiple Directorist search forms",
    "first_form_coords": true,
    "submitted_form_coords": false
  },
  "expected": {
    "mode": "investigation",
    "fix_surface": "child theme",
    "checks": ["request_fields", "published_listing_status", "duplicate_cityLat"]
  }
}
"

```

Initial eval cases:

- WP Reset negative safety wording.
- Hidden WordPress login route.
- Directorist duplicate geolocation fields.
- Draft listing should not be used as public search proof.
- Booking dropdown regression should inspect original flow first.
- Missing CSV export field should inspect export source and DB/meta key first.

Acceptance:

- New planner changes must pass support-case evals.
- Failed real tasks become new evals before new planner logic is merged.

Phase 8: Sidebar UX For Engineering Work

Goal:

Make long tasks understandable in the extension UI.

Add visible sections:

- Goal
- Current mode
- Evidence collected
- Risk gate
- Proposed patch
- Verification
- Files changed
- Next action

Do not show hidden chain-of-thought.

Show public trace:

```

““text
Observed current tab
Classified task as Directorist investigation
Fetched public search page
Found 4 duplicated search forms
Verified published listing coordinates
Backed up functions.php
PHP lint passed
Submitted child-theme patch
Logged-out verification passed
““

```

Acceptance:

- User can understand what happened without reading raw logs.
- The UI does not pretend to think; it reports real events only.

Phase 9: Python Core Integration

Goal:

Move orchestration into the Python-first ArafatAI core while keeping Node bridge available.

Python owns:

- agent routing
- long-term memory

- eval runner
- planning policy
- PR-gated self-improvement

Node bridge remains:

- lightweight sidebar bridge
- browser and Chrome-specific adapter
- optional dependency-free runtime for quick testing

Acceptance:

- Chrome sidebar can call either Node-only bridge or Python core bridge using the same JSON contract.

Phase 10: PR-Gated Self Improvement

Goal:

FLUID can propose its own improvement, but cannot silently merge it.

Workflow:

1. Detect repeated failure.
2. Write eval case.
3. Patch planner/tool code.
4. Run tests.
5. Create branch.
6. Prepare PR summary.
7. Human reviews and merges.

Hard rule:

No auto-merge.

First Implementation Order

Implement in this order:

1. Commit or review the existing WP Reset safety patch.
2. Add 'task-classifier.mjs'.
3. Add classifier tests for the known bad routes.
4. Add Evidence Store.
5. Add WordPress HTTP/session tools.
6. Add Patch Workflow helper.
7. Add Chrome CDP verification helper.
8. Add support-case evals.
9. Upgrade sidebar UI to show evidence and mode.
10. Start Python core integration only after Node bridge behavior is stable.

Concrete First Slice

Scope:

“text

Only improve routing and evidence.

No full desktop app yet.

No risky live-site automation yet.

“

Files to add/change:

- 'tools/sidebar-bridge-node/src/task-classifier.mjs'
- 'tools/sidebar-bridge-node/src/evidence-store.mjs'
- 'tools/sidebar-bridge-node/src/server.mjs'
- 'tools/sidebar-bridge-node/src/reasoner.mjs'
- 'tools/sidebar-bridge-node/test/task-classifier.test.mjs'
- 'tools/sidebar-bridge-node/test/evidence-store.test.mjs'
- 'extensions/chrome-sidebar/sidepanel.js'

Expected result:

- FLUID stops hallucinated direct fixes.
- FLUID says which mode it selected.
- FLUID records evidence before suggesting changes.
- Browser-only tasks stay fast.

Safety Policy

Blocked without explicit approval:

- reset
- delete
- publish
- submit production forms
- payment
- deploy
- merge
- credential changes
- database writes
- live theme/plugin edits

Allowed without approval:

- read public page
- read current tab snapshot
- navigate to user-provided URL
- inspect page text
- take screenshot
- query safe public REST endpoints
- run local tests
- read local files inside repo

Allowed after confirmation:

- edit child theme
- add Code Snippets
- run safe admin save
- submit test listing
- update local extension code
- commit/push repo branch

Verification Standards

For WordPress live fixes:

- Confirm active theme/plugin.
- Back up target file.
- Patch local copy.
- Lint PHP if PHP changed.
- Submit through admin or deployment path.
- Re-read live file.
- Check logged-out public page.
- Verify actual user flow or request result.

For extension/bridge fixes:

- Unit tests pass.
- At least one task log proves the old failure no longer routes incorrectly.
- Manual extension reload test passes.

For browser UI fixes:

- Desktop viewport screenshot.
- Mobile viewport screenshot.
- No text overlap.
- No hidden primary control.

What Not To Build Yet

Do not build these first:

- full desktop app UI
- autonomous live-site patching without confirmation
- auto-merge self-improvement
- broad WordPress database editor
- generic "fix everything" mode
- hidden reasoning display

These can wait until the evidence and safety loop is stable.

Success Definition

FLUID becomes useful when it can handle a case like this:

“text

User: Geo locating search returns no results on this WordPress/Directorist site.

FLUID:

- logs in if approved
- checks public form fields
- checks duplicate cityLat/cityLng
- checks listing status
- checks backend query requirements
- proposes child-theme patch
- backs up file
- lints patch
- applies patch
- verifies logged-out search returns listings
- explains exactly what changed

““

That is the engineering standard to implement toward.